Maven

What is a Build?

A **build** refers to the process of compiling source code into executable artifacts such as binaries (e.g., `.jar`, `.war`, `.exe`). The build process includes:

- **Compilation**: Converting source code into machine-readable code.
- **Packaging**: Bundling the compiled code into a deployable unit.
- **Testing**: Running unit tests to verify the correctness of the code.
- **Deployment**: Moving the build to the staging or production environment.

The build process can also include tasks like code quality checks, documentation generation, and versioning.

Purpose of Build Tools

Build tools automate the process of compiling, packaging, testing, and deploying code. They ensure consistency, reduce manual errors, and save time by automating repetitive tasks. The primary purposes are:

- **Automation**: Automates repetitive tasks like compiling, testing, and packaging.
- **Consistency**: Ensures the same process is followed each time, avoiding human error.
- **Efficiency**: Speeds up the development cycle by automating tasks.
- **Integration**: Facilitates the integration of various parts of a system (e.g., third-party libraries, databases).
- **Customization**: Allows for the definition of custom build processes based on project requirements.

Build Tools Ideology

The core ideology behind build tools is to **automate repetitive tasks** and ensure consistency. They allow developers to:

- Reproduce builds reliably.
- Integrate different components of a system (like libraries, third-party tools).
- Make builds portable, ensuring the same build process across different environments.

Build tools typically support a declarative or imperative approach:

- **Declarative**: Describes *what* should happen, not *how*.
- **Imperative**: Describes *how* tasks should be executed.

Evolution of Build Tools

- **Manual Build Scripts**: Initially, developers wrote custom shell scripts to compile code, run tests, and package artifacts.
- **Make**: The first widely used build tool. It allowed developers to define dependencies and actions in a `Makefile`.
- **Ant (Java)**: Introduced for Java projects, Ant provided more flexibility than Make but required detailed XML configuration.
- **Maven**: Introduced in 2004 as a standardized build tool, focusing on convention over configuration, simplifying dependency management, and integrating easily with project repositories.
- **Gradle**: Introduced as an alternative to Maven, providing a more flexible and efficient build system with support for both Groovy and Kotlin DSL.

Few Notable Build Tools

1. **Make**: A tool for managing build automation using makefiles, primarily for C and C++ projects.
2. **Ant**: A Java-based build tool, offering flexibility through XML configuration but requiring explicit configurations.
3. **Maven**: A Java-based build tool that emphasizes convention over configuration, centralizing dependency management and providing a standard build lifecycle.
4. **Gradle**: A modern build tool that supports Java, Kotlin, and Groovy. Gradle provides an advanced DSL for configuring builds and handles dependency management like Maven.
5. **MSBuild**: A Microsoft tool for building .NET projects.

Java-Based Build Tools

Java-based build tools are designed to manage Java projects' lifecycle, including compiling, packaging, and testing:

1. **Ant**: A flexible tool using XML-based configuration files to define tasks. It requires manual configuration for each task.
2. **Maven**: A more structured tool that follows the "convention over configuration" principle and automates the build lifecycle, dependency management, and project setup.
3. **Gradle**: Uses a Groovy-based DSL (or Kotlin) to define build processes and handles dependencies and build tasks efficiently.

Build Management

Build management refers to the process of automating and managing the lifecycle of software projects, including:

- **Dependencies**: Ensuring the correct libraries and tools are used.
- **Reproducibility**: Ensuring builds are repeatable across different environments.
- **Automation**: Automating the compilation, testing, packaging, and deployment processes.
- **Monitoring and Reporting**: Keeping track of build statuses, performance metrics, and logs.

Advantages of Build Tools

- **Automation**: Reduces manual work by automating build, test, and deployment tasks.
- **Consistency**: Ensures every team member runs the build in the same way, reducing errors and inconsistencies.
- **Speed**: Improves development speed by automating routine tasks.
- **Quality Control**: Integrates testing and validation into the build process, improving the overall quality of the code.
- **Dependency Management**: Handles complex project dependencies, ensuring that the right versions of libraries and tools are used.
- **Continuous Integration Support**: Many build tools are integrated with CI/CD tools like Jenkins, facilitating seamless integration of code changes and deployment.

Architecture of Maven

Maven is based on a **plugin-oriented** architecture where tasks (e.g., compile, test, package) are executed by plugins. Key components include:

1. **POM (Project Object Model)**: Central configuration file in XML format that defines the project's structure, dependencies, build settings, and plugins.
2. **Repository**: A central location where Maven stores project dependencies and build artifacts.
3. **Lifecycle**: A set of predefined steps (phases) for building and deploying an application.

Maven Build Life Cycle

Maven defines a standard build lifecycle with several phases:

1. **validate**: Validate the project structure.
2. **compile**: Compile the source code.
3. **test**: Run unit tests on the compiled code.
4. **package**: Package the compiled code into a distributable format (e.g., JAR, WAR).
5. **verify**: Verify the packaged code (integration tests, validation).
6. **install**: Install the packaged code into the local Maven repository.
7. **deploy**: Deploy the packaged code to a remote repository for sharing with other developers or projects.

Each phase has predefined goals (tasks) that are executed when the phase is triggered.

Maven Directory Structure

Maven defines a standard directory layout for project files:

```
src/
  main/
    java/             -> Java source code
    resources/        -> Non-code resources (e.g., config files)
  test/
    java/             -> Unit tests
    resources/        -> Test resources
target/               -> Build output (e.g., JAR, WAR)
pom.xml               -> Project configuration file
```

The directory structure ensures consistency across Maven projects, making it easier to navigate and understand the layout.

Maven Repositories

Maven repositories are storage locations for project artifacts (e.g., JAR files). There are three types:

1. **Local Repository**: Stored on the developer's machine. When Maven downloads dependencies, it caches them here.
2. **Central Repository**: A remote repository (usually hosted by Maven or other public organizations) that contains common open-source libraries.
3. **Remote Repository**: Repositories hosted by organizations or other third parties where private or proprietary artifacts are stored.

POM.xml (Project Object Model)

The **pom.xml** file is the core configuration file for a Maven project, containing:

- **Project information**: Group ID, artifact ID, version, packaging type.
- **Dependencies**: A list of external libraries required by the project.

- **Plugins**: Configuration for build plugins to handle tasks like compilation, testing, and packaging.
- **Build settings**: Define build directory, final name of the artifact, etc.
- **Profiles**: Different configurations for various environments (e.g., development, production).

Sample `pom.xml`:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>myapp</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

Multi-Module Project (Overview)

A **multi-module project** in Maven is a project with multiple sub-projects (modules) that share common settings or dependencies. It allows for modular development while maintaining a centralized build configuration.

Key Features:

- **Parent POM**: The root POM that manages common configurations and dependencies.
- **Module POMs**: Each sub-project has its own `pom.xml` that can inherit configurations from the parent POM.

Example of a multi-module structure:

```
parent-project/
  pom.xml              -> Parent POM
  module1/
    pom.xml            -> Module 1 POM
  module2/
    pom.xml            -> Module 2 POM
```

The parent POM can manage shared dependencies, build configurations, and plugin versions, reducing duplication.

my notes

>java based application that helps manage

maven automatically upgrade the dependencies in your project

>it has vase huge free dependences

https://maven.apache.org/plugins/

here you can check multiple dependencies

it does lots of work automatically

tool installation

java

maven

git

any ide intellij

to install java

https://www.oracle.com/in/java/technologies/downloads/

https://www.oracle.com/in/java/technologies/downloads/#jdk23-windows

download it through exe

to set path for the file

go inside you c > these pc>c>programs>java>sdk>bin

copy path and paste it in env

and you are done you can check it from cmd

>java –version

```
C:\Users\Admin>java --version
java 23.0.1 2024-10-15
Java(TM) SE Runtime Environment (build 23.0.1+11-39)
Java HotSpot(TM) 64-Bit Server VM (build 23.0.1+11-39, mixed mode, sharing)

C:\Users\Admin>
```

now lets install maven

search on google download maven

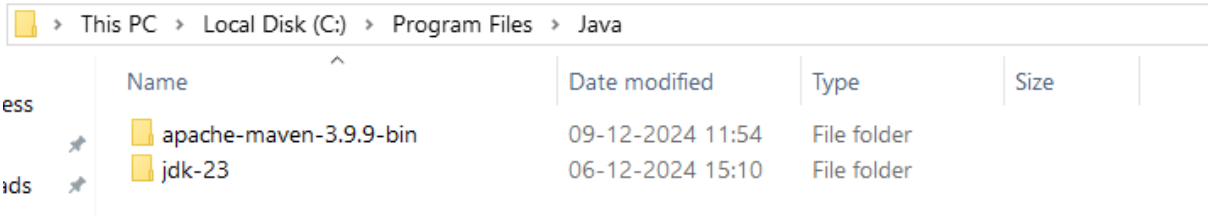
https://maven.apache.org/download.cgi

download binary zip file

now unzip it

now curt it and paste in inside java

>pc>program and java paste it



now go inside you maven and bin copy it and paste it inside env var

add a new

var name JAVA_HOME

C:\Program Files\Java\jdk-23

```
C:\Users\Admin>mvn -v
Apache Maven 3.9.9 (8e8579a9e76f7d015ee5ec7bfcdc97d260186937)
Maven home: C:\Program Files\Java\apache-maven-3.9.9-bin\apache-maven-3.9.9
Java version: 23.0.1, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-23
Default locale: en_IN, platform encoding: UTF-8
OS name: "windows 10", version: "10.0", arch: "amd64", family: "windows"
```

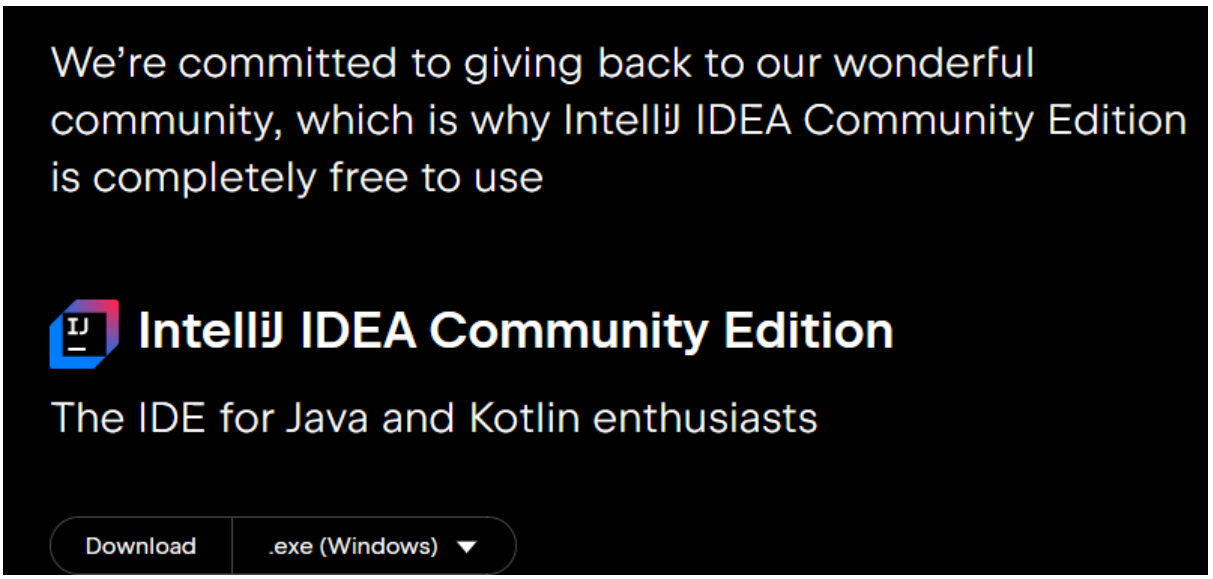
now install git

```
C:\Users\Admin>git -v
git version 2.47.1.windows.1

C:\Users\Admin>
```

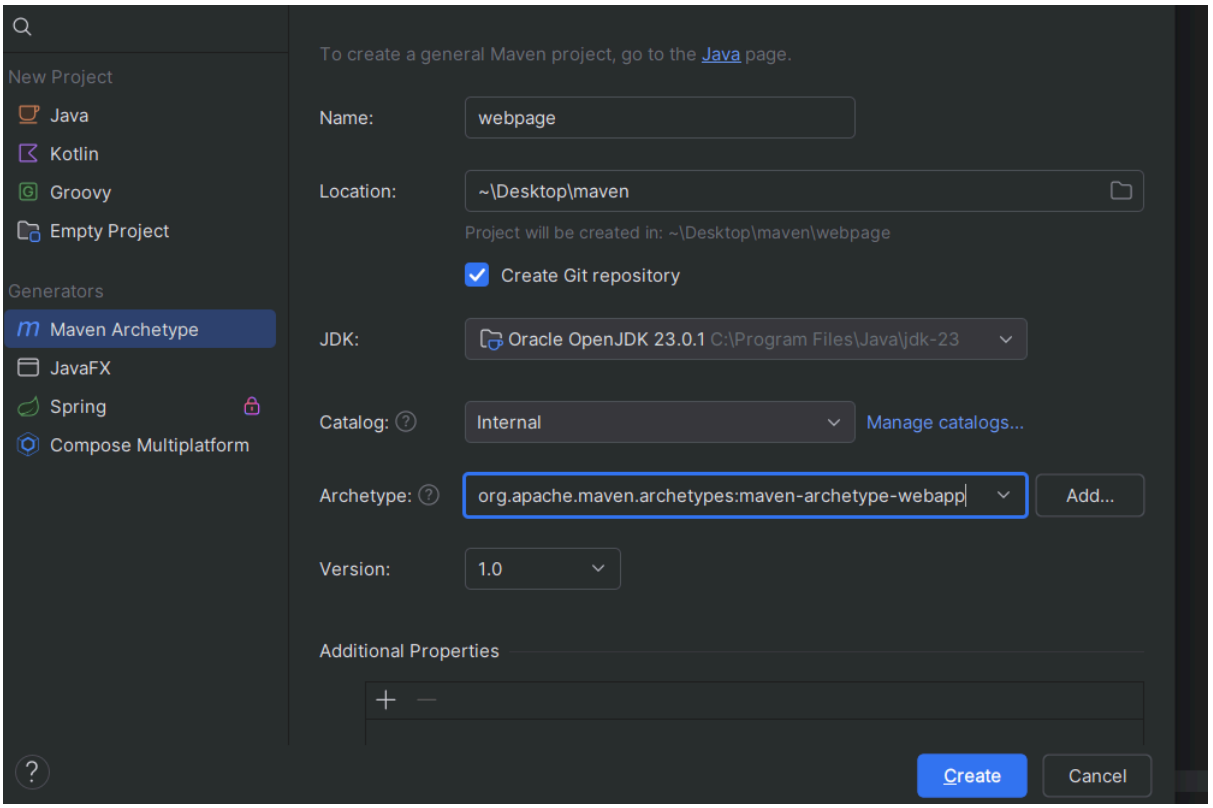
now install intellij

https://www.jetbrains.com/idea/download/?section=windows



creating a web application with maven

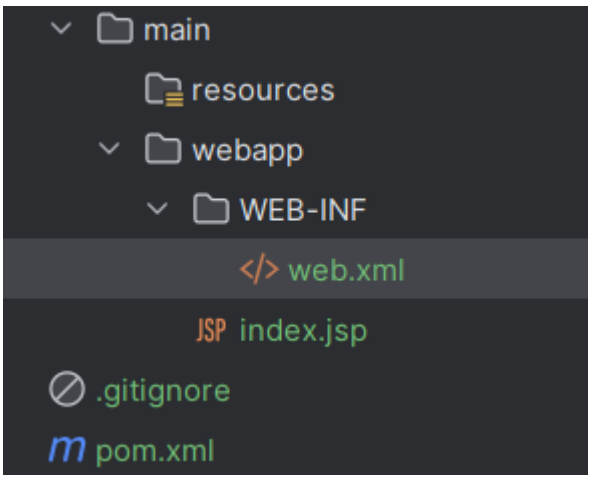
for creating a web application in maven you need archetype



click on create

it will install all the dependency that is required

these is the main file of your project



inside index.jsp you can modify as you want

got to maven >LIFECYCLE >PACKAGES

click on it

so it will create a target file

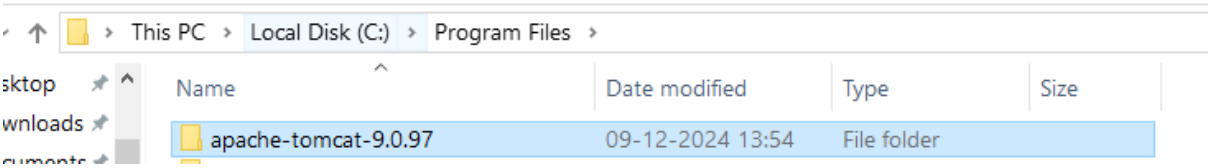
inside it will create a war file that will can deploy it in the tomcat server

now lets install apache tomcat to deploy the web app

https://tomcat.apache.org/download-90.cgi

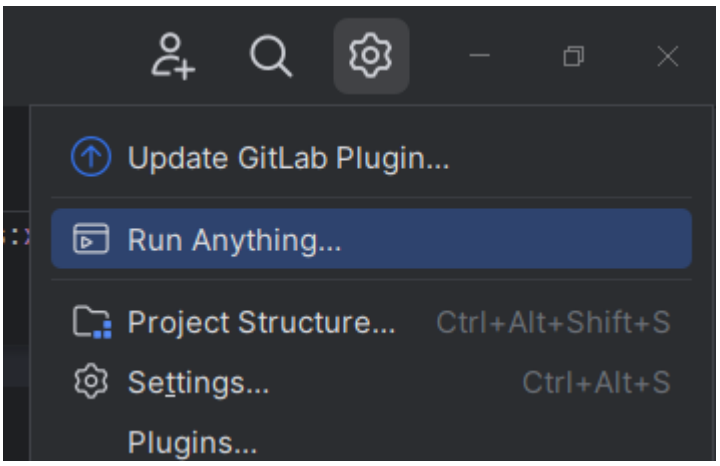
download it form it website

go inside it and copy the folder and paste in inside your program files

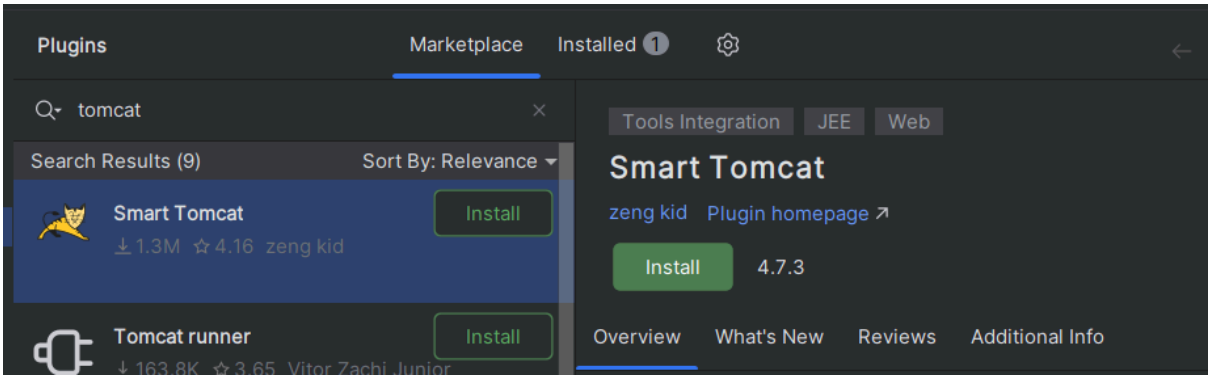


now now got to you project

click on plugins

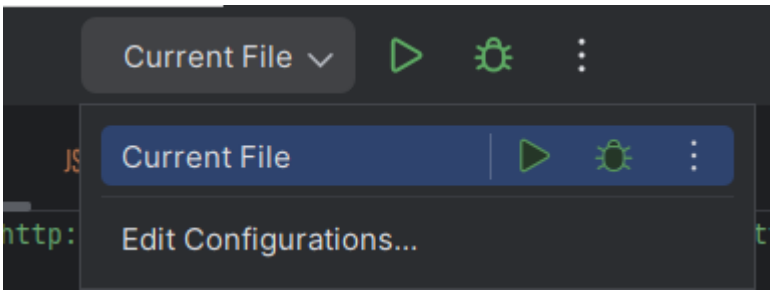


and search smart tomcat



install it

click on apply okay



click on edit configuration

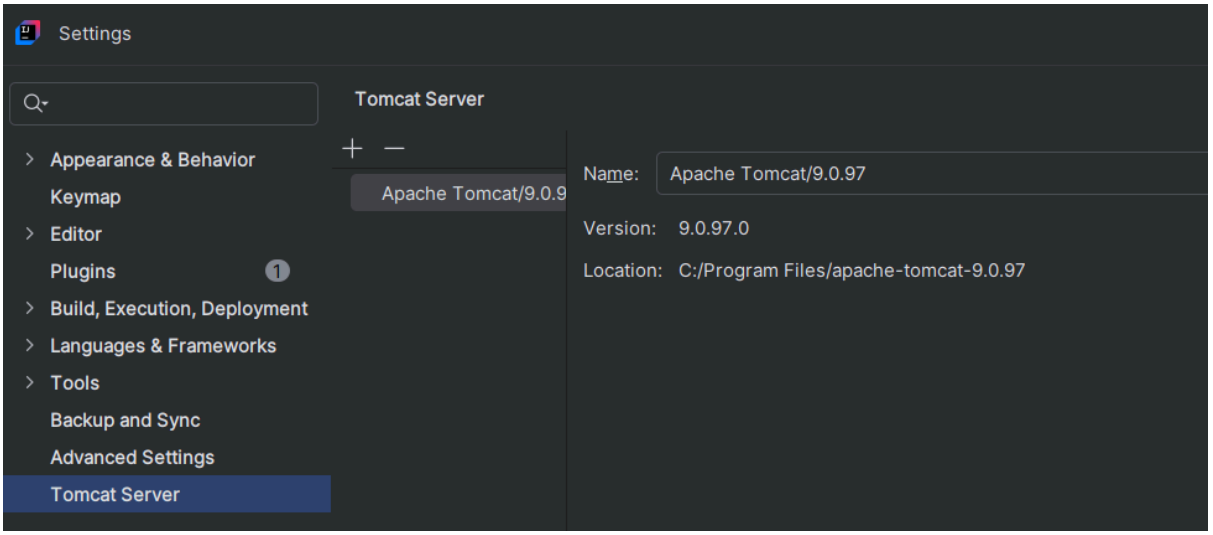
click on plus icon

you will see an smart tomcat option

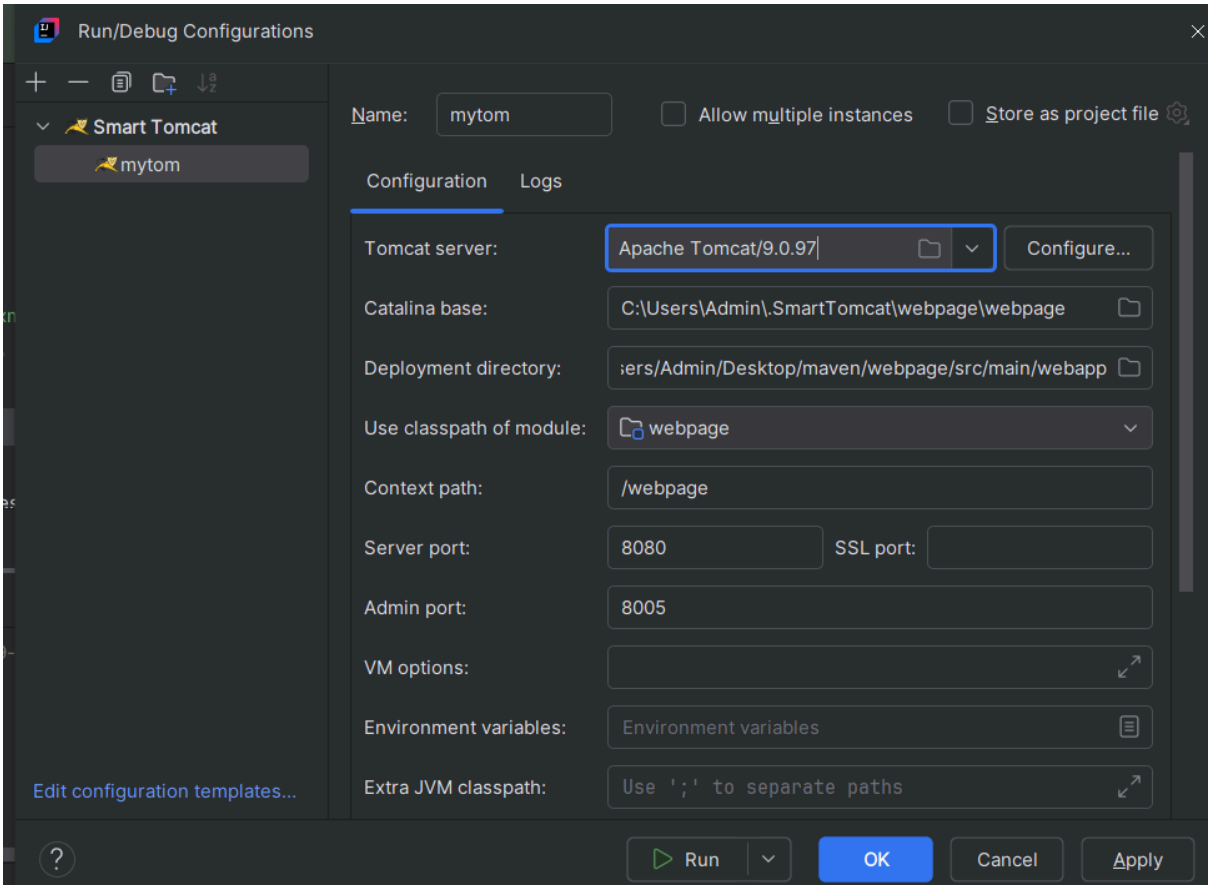
if you see tab is empty click on configure

click on plus

now give the path of your apache server



click on apply and okay



now you will your tomcat here

now go to maven and click on clean

compile

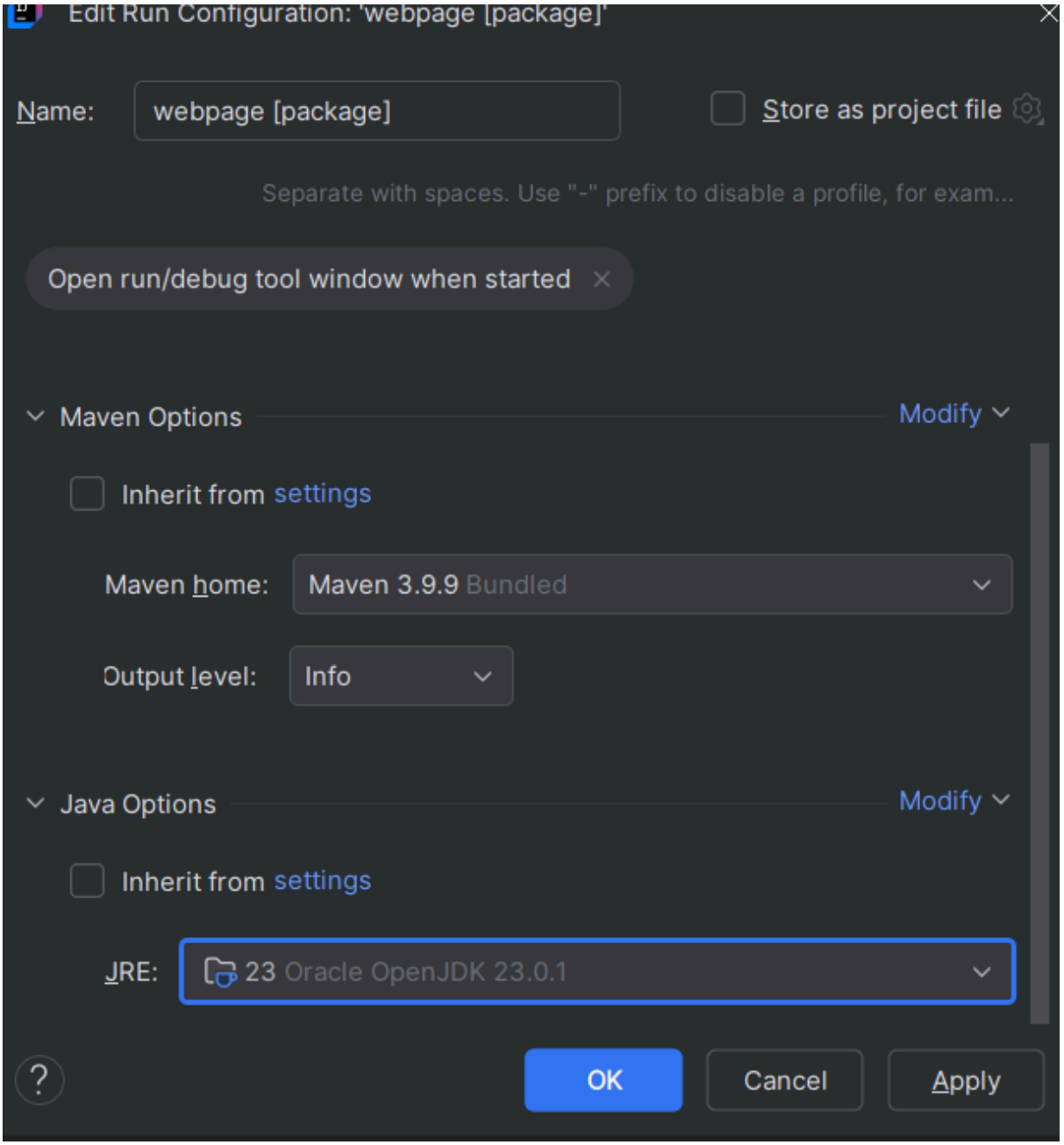
and package

```
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.072 s
[INFO] Finished at: 2024-12-09T14:29:23+05:30
[INFO] ------------------------------------------------------------------------

Process finished with exit code 0
```

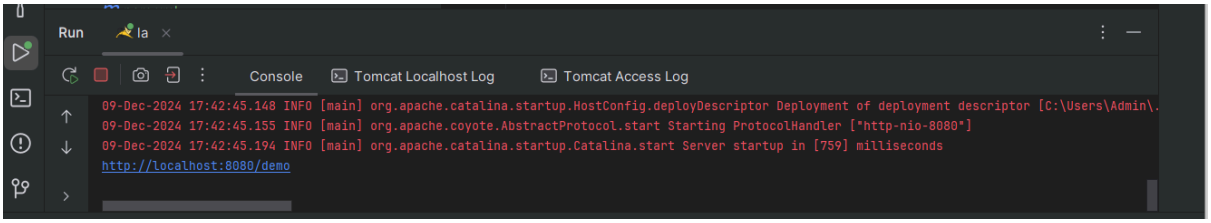
now click on run button

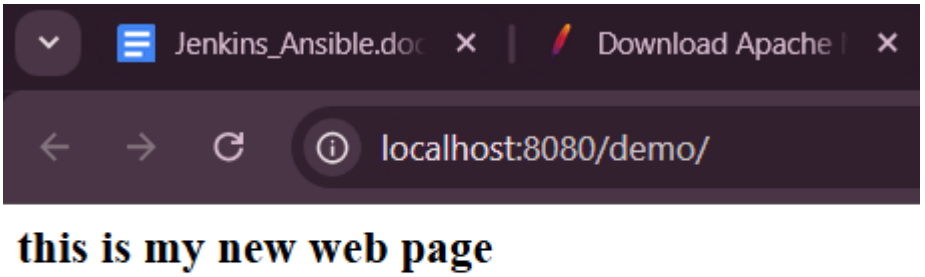
click on modify run configuration



select it run apply

project





and it's done

create and manage multi module

>create a simple maven project

with java and maven

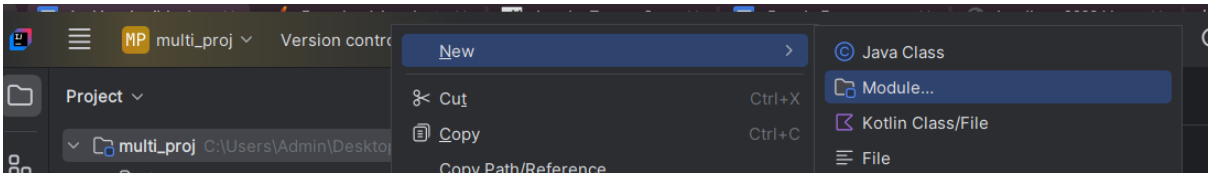
for multi module

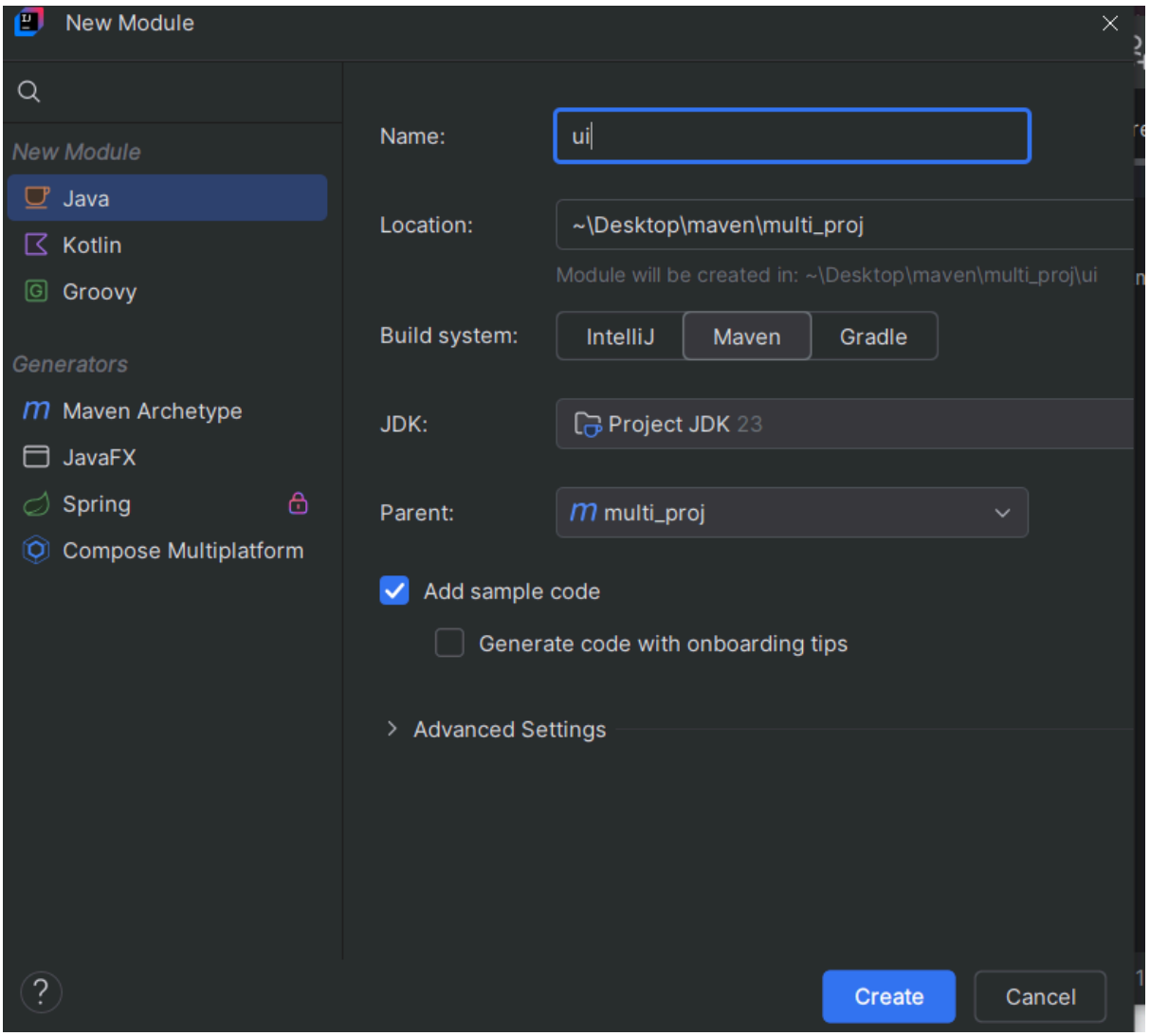
>>util

>>core

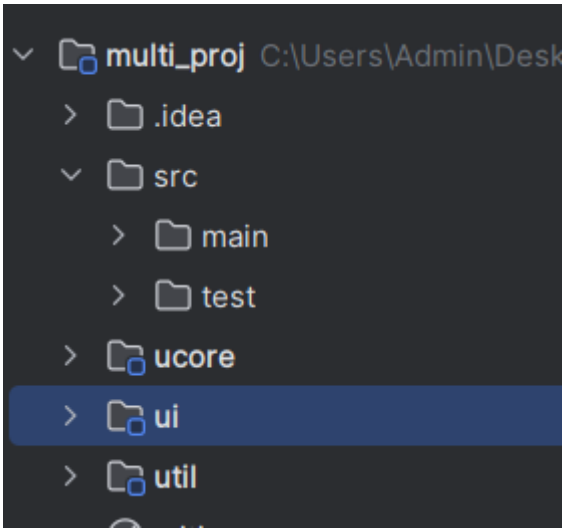
>>ui

right click on your project >module



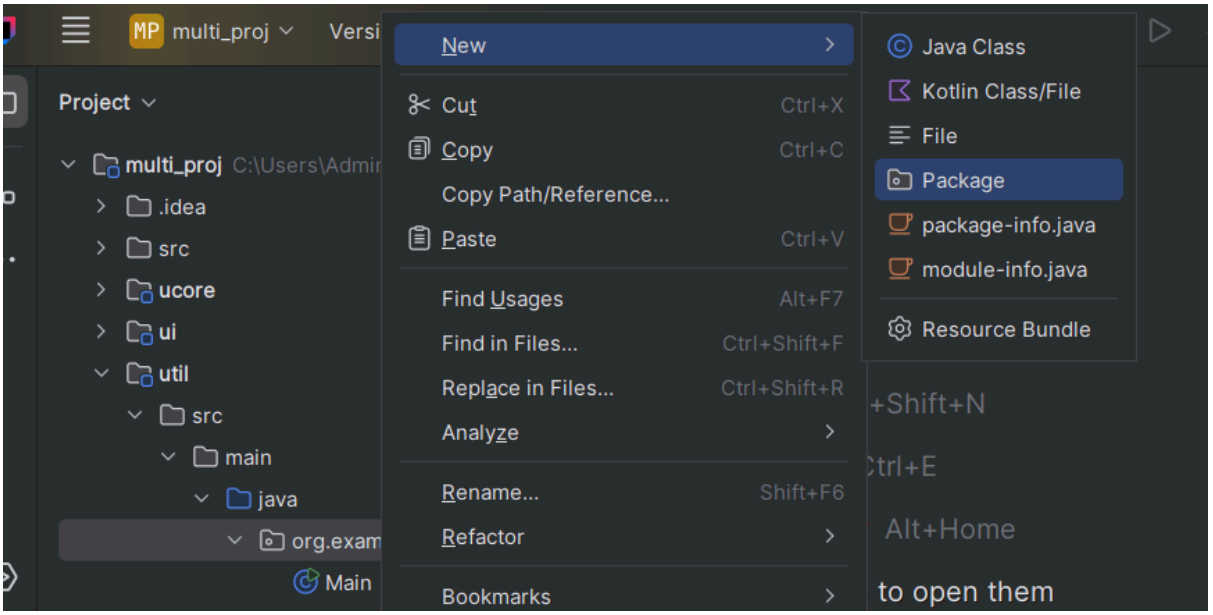


it will create a model in your project

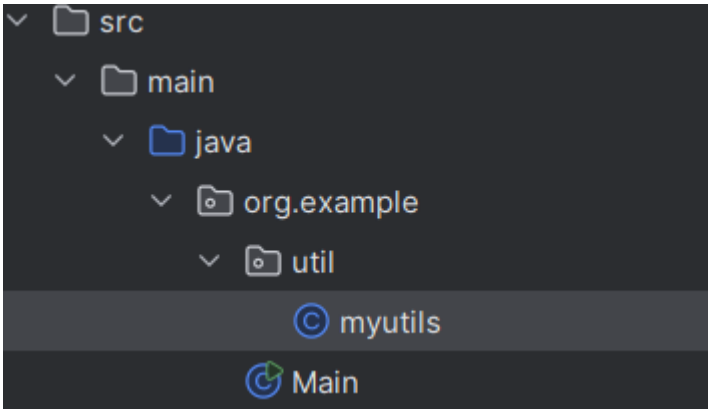


to access one module from other module

go inside util



now create one java class ini t



in the utils file add

```
package org.example.util;


public class myutils {
    public static void PrintName(String name){
        System.out.println("hello there "+name+" Welcome!!!");
    }
}
```

now if you want to access it from main class

then u need to add the dependency in your core module

go inside core pom.xml

```
<dependencies>
    <dependency>
```

```
        <groupId>org.example</groupId>
        <artifactId>util</artifactId>
        <version>1.0-SNAPSHOT</version>
    </dependency>
```

add these code in your mane package form which you want to access it

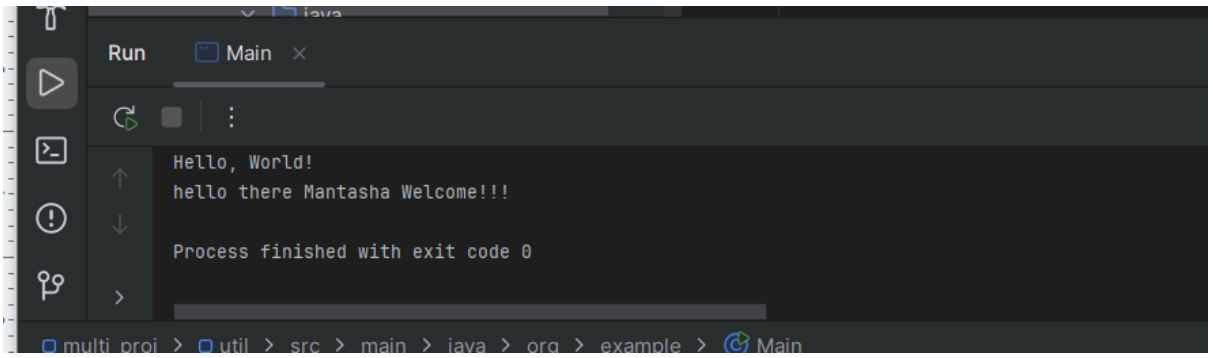
if you add dependency then you need to load the maven

now goto main class of ucore

```
package org.example;
import org.example.util.myutils;


public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
        myutils.PrintName("Mantasha");
    }
}
```

now click on run and you wll see hello world



and you are done!!!